

Reliability in AI for Mathematics: An Assurance-Chain Framework and a Corpus Audit

Songpengfei*

October 5, 2026

Abstract

Large language models now produce mathematical text that is fluent, plausible, and sometimes wrong. Work on making such output trustworthy has converged on three families of mechanisms — sampling with an external verifier, formalization into a proof assistant, and contamination-controlled evaluation — but these are usually presented as competing options on a single axis of quality. We argue that this framing is mistaken, and we propose an alternative.

We model the path from an informal mathematical claim to a machine-checkable verdict as a four-link *assurance chain*: informal claim $\rightarrow$ reduction $\rightarrow$ formal object $\rightarrow$ verdict. Each of the three paradigms intervenes at a different link, and each therefore leaves behind a residual risk of a *different kind*: the verifier paradigm leaves the reduction jump unverified, the formalization paradigm risks proving the wrong theorem correctly, and the evaluation paradigm risks construct validity. Because the residual risks are incommensurable, “which paradigm is best” is not a well-posed question; the useful question is which residual risk a given deployment can tolerate. We derive an operational twelve-item checklist from the framework.

We then turn the framework on our own research process. Constructing the literature review for this paper, we first generated candidate references from model memory and then verified them against three independent bibliographic indexes. Of 31 candidates, three (9.7%) were genuinely wrong in ways that a title-only check would have accepted, including one case with a well-formed but entirely unrelated arXiv identifier. In a second study we measure how far the *reproducible evaluation* paradigm is actually practised in the corpus: only 13 of 26 arXiv-indexed works (50% after human adjudication) ship a public official artifact, and automated detection alone recovers just 8 of those 13 (precision 80%, recall 62%). We report these results as evidence for the paper’s own thesis — that generation without a verification layer fails in characteristic, measurable ways — and we are explicit about the limits of both studies.

Keywords: AI for mathematics, reliability, formal verification, automated theorem proving, benchmark validity, reproducibility, citation integrity.

1 Introduction

A language model asked to prove a theorem will almost always produce something. It will be grammatical, use the right notation, cite plausible intermediate steps, and arrive at the stated conclusion. It will also, at a rate that has proved stubbornly resistant to scale alone, be wrong [Cobbe et al., 2021, Lightman et al., 2023, Huang et al., 2023]. The failure is not a failure of fluency. It is a failure of *verifiability*: the output is a string in a natural language, and natural-language strings do not come with a procedure that decides whether they are correct.

The field’s response has been to build such procedures, and it has built them in three recognisably different ways.

*Submission for the EduSeed challenge C2 -- AI for Math paper.

The first samples many candidate solutions and scores them with a learned or symbolic *verifier*, keeping the highest-scoring one [Cobbe et al., 2021, Lightman et al., 2023]. The second translates the problem into a *formal* system — Lean, Coq, or an executable program — where a kernel, not a model, decides correctness [Wu et al., 2022, Jiang et al., 2022, Yang et al., 2023]. The third concerns not the answer but the *measurement*: it asks whether the benchmark on which the claim of correctness rests is itself sound [Zhou et al., 2023, Sainz et al., 2023, White et al., 2024].

These three families are frequently compared as if they were candidates for a single job. We argue that they are not, and that treating them as such obscures the one thing a practitioner most needs to know: *what remains unverified*. A verifier that scores answers highly has said nothing about whether the question was translated correctly. A proof assistant that accepts a proof term has said nothing about whether that term formalises the theorem the user had in mind. A benchmark on which a system scores well has said nothing about whether the benchmark measures the capability of interest.

Contribution. This paper makes four contributions.

1. **A framework** (§3). We decompose the path from informal claim to verdict into four links — an *assurance chain* — and show that each of the three paradigms defends a different, proper subset of it. This yields a *residual-risk matrix* that makes the residual failure mode of each paradigm explicit and comparable without pretending it can be eliminated.
2. **A taxonomy** (§2). We organise the relevant literature into the three paradigms and, in doing so, give a reading of the recent negative results on self-correction [Huang et al., 2023, Stechly et al., 2024] as a statement about link structure rather than as a surprising empirical fact.
3. **A checklist** (§6). Twelve questions, each traceable to a specific link in the chain, that a team can apply before claiming a system is reliable.
4. **An empirical audit** (§4–§5). We measure the failure rate of the citation-generation step in this paper’s own literature review (9.7%), and the prevalence of public artifacts in the surveyed corpus (50%, against 27% recovered by an automated screen alone).

The fourth contribution is deliberately self-referential. The framework says that generation without an independent verification layer fails in characteristic ways; a literature review produced by a language model is exactly such a generation step. We therefore report the audit of our own bibliography as data, not as an aside.

Scope. We make no claim that the framework is complete, nor that the audit is representative. Both limitations are stated in §7.2.

2 Related work

Formal mathematics and autoformalization. Learning-based theorem proving begins with the observation that a proof assistant can be treated as an environment: Polu and Sutskever [2020] train a language model to propose proof steps and use the assistant to filter them, and Han et al. [2021] extend this by co-training on intermediate proof state rather than only on complete proofs. Wu et al. [2022] identify the step that makes the approach possible at all — *autoformalization*, the translation of informal statements into a formal language — and show it is the fragile part. Jiang et al. [2022] use an informal proof as scaffolding for a formal prover, and Yang et al. [2023] add retrieval over a large formal corpus. Xin et al. [2024] and Ren et al. [2025] scale the data and the search; Lin et al. [2025] release a frontier open model. Outside the

interactive-prover setting, [Trinh et al. \[2024\]](#) pair a symbolic deduction engine with a language model to solve olympiad geometry, and [Romera-Paredes et al. \[2023\]](#) search over *executable* programs scored by an automatic evaluator. The underlying proof language is developed by [de Moura and Ullrich \[2021\]](#).

A parallel line reduces natural-language reasoning to *programs* rather than proofs: [Gao et al. \[2022\]](#) and [Chen et al. \[2022\]](#) both delegate computation to an interpreter, and [Schick et al. \[2023\]](#) has the model learn to call tools itself.

Verifiers and search. [Cobbe et al. \[2021\]](#) introduce GSM8K together with the now-standard recipe of sampling many solutions and re-ranking with a trained verifier. [Lightman et al. \[2023\]](#) move the verification granularity from the final answer to individual reasoning steps, which is the strongest available positive evidence for the verifier paradigm. The negative evidence is equally important: [Huang et al. \[2023\]](#) show that a model asked to revise its own reasoning without external feedback does not reliably improve, and [Stechly et al. \[2024\]](#) show that the self-verification signal is not trustworthy even when the model is asked only to judge. [Valmeekam et al. \[2023\]](#) make the same point for planning, and [Kambhampati et al. \[2024\]](#) turn it into an architecture: the language model generates, an external critic verifies, and the loop is driven by the critic.

Evaluation and reproducibility. [Hendrycks et al. \[2021\]](#) provide the MATH benchmark that most of the above is measured on. [Zhou et al. \[2023\]](#) survey the ways a model can appear better than it is through benchmark artefacts; [Sainz et al. \[2023\]](#) turn contamination into a quantity to be estimated per benchmark; [White et al. \[2024\]](#) respond with a continuously refreshed benchmark, and [Rein et al. \[2023\]](#) with one designed to resist lookup. [Wang et al. \[2023\]](#) and [Lu et al. \[2024\]](#) situate these concerns in the broader programme of automating science.

Positioning. Our framework is closest in spirit to the three-layer decomposition of [Chen and Jiang \[2026\]](#), which separates generation, semantic reduction and verification, and identifies the reduction layer as the bottleneck. That decomposition is the substrate of §3; we adopt it, and we differ from it in three ways. First, we add the evaluation layer as a concern that operates *before* the chain rather than as another link in it. Second, and centrally, we map the three literature paradigms onto the chain and show that each leaves a residual risk of a distinct kind — the decomposition identifies where the difficulty is, but not that the difficulty is relocated rather than removed by each paradigm. Third, we turn the framework into an operational checklist and into an audit procedure that we then execute on our own review process.

3 The assurance chain

3.1 Four links

We model the production of a machine-checkable mathematical result as a chain of four links. The decomposition follows [Chen and Jiang \[2026\]](#), with the terminology made explicit for the purposes of this paper.

L_1 — **Informal claim.** A mathematical assertion in natural language, together with whatever argument accompanies it. Undecidable in principle: deciding whether a natural-language string expresses a true proposition is not a well-posed computational problem.

L_2 — **Reduction.** A mapping from L_1 to an object in a formal language: a proof term, a program, a constraint system. This is the *semantic reduction* step. It is not a bijection —

natural language is ambiguous, under-typed, and not invertible — and no general-purpose correctness criterion exists for it.

L_3 — **Formal object.** The artefact produced by L_2 , together with an explicit specification of what it is supposed to establish. Unlike L_1 , this object is syntactic: it can be checked.

L_4 — **Verdict.** The decision procedure applied to L_3 : a proof kernel, a satisfiability solver, a numerical evaluator, or a learned verifier. This is the only link with an unambiguous notion of correctness.

The chain is only as trustworthy as its weakest link, and the links are not independent: a verdict at L_4 is a statement about L_3 , and L_3 is a statement about L_1 only insofar as L_2 was faithful. This is the structural reason why a correct verdict does not imply a correct answer, and it is the fact that the three paradigms each address only in part.

3.2 Three paradigms, three links

V — Verifier-in-the-loop. Sample k candidate reductions; score them; keep the best. The verifier operates at L_4 and may be learned [Cobbe et al., 2021, Lightman et al., 2023] or symbolic [Kambhampati et al., 2024].

F — Formalization. Force L_2 to produce an object in a language whose L_4 is decidable, and require the verdict to come from that decision procedure. The paradigm thus defends L_2 , L_3 and L_4 simultaneously [Wu et al., 2022, Jiang et al., 2022, Yang et al., 2023].

R — Reproducible evaluation. Constrain the process *before* L_1 : ensure that the claim being tested is the claim one intends to test, and that the measurement is not contaminated [Sainz et al., 2023, White et al., 2024, Rein et al., 2023].

3.3 Residual risk

The substantive claim of this paper is that the three paradigms leave behind residual risks that are not merely different in degree but different in kind, and that this is why they compose rather than compete. Table 1 states the claim; Figure 1 shows it.

Adopting the terminology of Chen and Jiang [2026], call the failure at L_2 a *reduction jump*: the informal statement and its formal image are not related by any checkable procedure, so a discrepancy between them is invisible to everything downstream. The V paradigm, operating at L_4 , cannot detect it; the F paradigm can detect some of it, but only to the extent that the formalization step is itself checked, which in practice it is not — the prover proves whatever it was given. The R paradigm does not bear on the jump at all.

3.4 Consequences

Three consequences follow, and they organise the rest of the paper.

(i) **“Which paradigm is best?” is ill-posed.** The paradigms are not points on one axis. A verifier can be arbitrarily good and still score a faithfully-solved wrong statement highly; a formalization can be kernel-checked and still prove the wrong theorem; a benchmark can be contamination-free and still measure the wrong thing. Comparisons of the form “formalization beats verification” therefore require an implicit assumption about which residual risk is tolerated, and are usually stated without it.

Table 1: The residual-risk matrix. Each paradigm defends a proper subset of the assurance chain and leaves a residual failure mode that the other two address only partially. Reading the table row-wise answers the practical question “what is still unverified if I adopt this?”

	Defends	Residual risk	Why the others do not close it
V	L_4	The reduction jump $L_1 \rightarrow L_2$ is unverified, and the verifier is itself a learned model sharing the generator’s distribution.	F can in principle check the reduction, but only by formalising it, which moves the problem rather than solving it. R says nothing about it.
F	$L_2\text{--}L_4$	<i>Proving the wrong theorem correctly.</i> A kernel accepts a proof of the formalised statement; it has no opinion on whether that statement is the one intended. Coverage is also partial: most mathematics is not formalised.	V has no access to the formal statement. R does not inspect statements at all.
R	before L_1	<i>Construct validity.</i> A benchmark is an operationalization of a capability, and a system can improve on the operationalization without improving on the capability.	Neither V nor F can detect a saturated or contaminated benchmark: they validate answers, not the choice of question.

(ii) **The negative results on self-correction are a link-structure result.** [Huang et al. \[2023\]](#) and [Stechly et al. \[2024\]](#) show that a model cannot supply its own L_4 . In the terms of Table 1, a self-verifier does not satisfy the precondition of V — that the verifier be external to, and better on the target distribution than, the generator. The empirical finding is thus predicted by the framework rather than incidental to it.

(iii) **Verification quality degrades when the verifier is itself generated.** If a verifier is produced by the same kind of process as the generator, the two share failure modes, and the verifier’s errors are correlated with the generator’s. §5 reports a measurement of exactly this effect in a setting where the stakes are low and the ground truth is cheap: bibliography construction.

4 Methodology

We now apply the framework to a concrete generation step: the construction of this paper’s literature review. The step is a pure L_1 operation — a language model emits plausible-looking bibliographic entries, with no L_2 reduction and no L_4 verdict available at generation time.

4.1 Corpus construction

A candidate list of 31 references was written from model memory with no retrieval, deliberately, so that the failure rate of that process could be measured rather than assumed. The candidate list is preserved verbatim (`pipeline/refs_candidates.json`, revision v1; the verification output for it is archived as `citation_verification_v1_from_memory.json`).

Each candidate was then submitted to three independent bibliographic indexes:

- **OpenAlex** (api.openalex.org), as the primary index, chosen for its coverage of arXiv preprints;
- **Crossref** (api.crossref.org), the DOI registration agency, as an independent publishing-side record;

The four-link assurance chain and where each paradigm intervenes

No single paradigm covers the whole chain: residual risk is not removed, only moved.

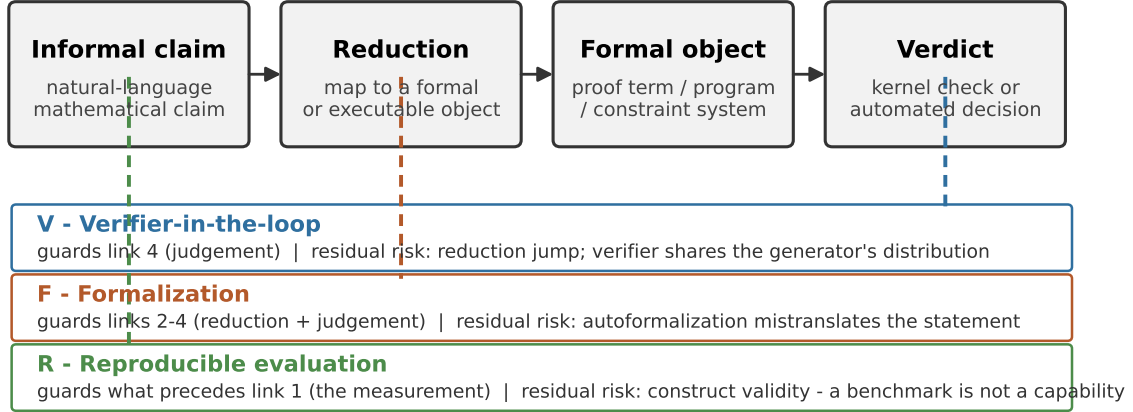


Figure 1: The four-link assurance chain, where each paradigm intervenes, and the residual risk it leaves behind. No single paradigm covers the whole chain.

- the **arXiv abstract page** for the exact identifier, as a direct existence check on the identifier itself.

A candidate is accepted only if all three of the following hold: the best matching title reaches a normalised similarity of ≥ 0.90 ; the first author’s surname agrees; and the publication year agrees within ± 1 . The year tolerance is explicit and is recorded in the machine-readable output; it exists because preprint and proceedings years routinely differ (Yang et al., 2023: arXiv 2023, NeurIPS 2023).

Candidates that fail are not silently dropped. They are adjudicated by hand, and every correction is recorded in `pipeline/refs_corrections.json` with the wrong value, the corrected value, and the evidence that decided it.

4.2 Study A: reliability of the citation-generation layer

Study A reports how the 31 candidates resolve: how many the automated check flags, how many of those flags survive human adjudication, and what the failure modes are. The quantity of interest is the number of entries that are *genuinely wrong in a way a weaker check would have accepted* — the relevant question, because a check that only confirms that an identifier resolves will pass a well-formed but unrelated identifier.

4.3 Study B: artifact availability in the corpus

Study B asks how far the R paradigm is practised, not merely advocated, in the corpus. For each entry with an arXiv identifier we measure whether a public *official* artifact exists — a repository owned by the authors that the description or README ties to the paper.

Two signals are collected. First, the abstract is fetched from the arXiv abstract page and scanned with a fixed set of patterns for an explicit release statement. Second, the GitHub search API is queried with the paper’s project name, and the top result is accepted if the repository name matches the project name at ≥ 0.90 normalised similarity or the repository description matches the paper title at ≥ 0.70 .

Neither signal is authoritative. The automated screen produces both false positives (a repository whose name shares a common word with the paper) and false negatives (an official

Figure 2. Reliability audit of the citation-generation layer - error rate 9.68%

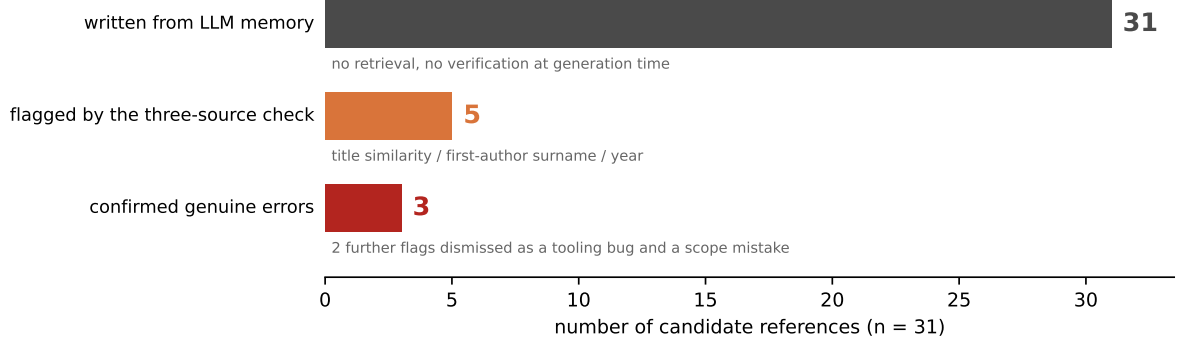


Figure 2: Study A. Thirty-one candidate references written from model memory without retrieval; the automated three-source check flags five; three are confirmed genuine errors after adjudication.

repository whose README cites the paper without restating its title). We therefore adjudicate all 26 rows by hand, fetching each candidate repository’s description and README and recording the evidence verbatim in `pipeline/artifact_adjudication.json`. Both the automated and the adjudicated numbers are reported, because the gap between them is itself a result.

4.4 Reproducibility

Every number in §5 is produced by the scripts in `pipeline/`: `01_verify_refs.py` (verification), `02_build_bib.py` (bibliography generation), `03_corpus_audit.py` (both studies) and `04_make_figures.py` (figures). The bibliography is a build artefact, not a hand-written file, and `02_build_bib.py` emits a SHA-256 of its output so that a reader can confirm they reproduced it byte for byte. The figures read their numbers from the JSON that `03_corpus_audit.py` writes, so a figure cannot drift out of sync with a table.

5 Results

5.1 Study A: the citation-generation layer

Of the 31 candidates written from memory, the three-source check flagged five. Human adjudication resolved those five into three genuine errors, one false alarm caused by a defect in our own checking script, and one entry that was mislabelled by us rather than mis-stated by the model. The genuine error rate is therefore $3/31 = 9.7\%$ (Figure 2, Table 2).

The failure modes are worth separating, because they call for different defences.

Wrong identifier (entry 1). The model produced a well-formed arXiv identifier that resolves to a real paper — about precision–recall curves in classification, entirely unrelated to theorem proving. It also produced a plausible title, apparently conflating the first DeepSeek-Prover paper with the subtitle of its successor. This is the failure mode that a weakest-form check (“does the identifier resolve?”) cannot detect at all, and it is the reason our protocol compares titles rather than merely confirming existence.

Word-level alteration (entry 2). The model wrote *contamination-free* where the paper says *contamination-limited*. The direction of the error is not random: it substitutes a stronger claim for the author’s more careful one. Title similarity was 0.92 — high enough to look acceptable in a coarse review, and only separable by word-level comparison.

Keyword substitution (entry 3). *Program verification* replaced *theorem proving*: a term from the same neighbourhood, more common in the literature, but not the author’s.

Table 2: The three genuine errors in the memory-written bibliography. The third column gives the value the model produced; the fourth gives the verified value. Entry 1 is the most instructive: the fabricated arXiv identifier is syntactically valid, resolves successfully, and points to an unrelated paper.

#	Field	Generated (wrong)	Verified (correct)
1	Title	<i>DeepSeek-Prover: Advancing Theorem Proving in LLMs through Subgoal Decomposition</i>	<i>DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data</i>
	arXiv ID	2308.08381 (resolves to an unrelated paper on precision–recall curves)	2405.14333
	Year	2023	2024
2	Title	LiveBench: A Challenging, Contamination-Free LLM Benchmark	LiveBench: A Challenging, Contamination-Limited LLM Benchmark
3	Title	Goedel-Prover: A Frontier Model for Open-Source Automated <i>Program Verification</i>	Goedel-Prover: A Frontier Model for Open-Source Automated <i>Theorem Proving</i>

Table 3: Automated detection scored against human adjudication, across all 26 measured entries (Table 4 covers the 24 that belong to a paradigm). An entry counts as detected if either automated signal fires. This is the paper’s thesis measured on its own tooling: the reduction from “repository page” to “verdict” is the unreliable part, in both directions at once.

Automated screen vs. adjudication	n
True positives (flagged, confirmed official)	8
False positives (flagged, not official)	2
False negatives (not flagged, is official)	5
Precision	80.0%
Recall	61.5%

The false alarm, and what it says. One entry was flagged purely because our own script failed to parse the submission date from the arXiv abstract page, leaving the year field empty and the year check unsatisfiable. The entry was correct throughout. We report this because it is the same phenomenon the paper is about, one level up: *the checking apparatus is itself a generation step that needs checking*. An unfalsifiable automated verdict is not a verdict.

A second-order observation. The three errors were all caught by the automated screen, but only because the screen compared titles non-exactly. Had the screen required only that each reference resolve, all three would have passed: entry 1’s identifier resolves, and entries 2 and 3 have correct identifiers and merely wrong titles. The strength of the check, not the existence of a check, determines what it catches.

5.2 Study B: artifact availability

Four of the 30 accepted references (the journal and proceedings articles without an arXiv identifier) fall outside Study B’s scope, leaving 26 measured entries.

The abstract-wording screen alone found an explicit artifact release statement in 7 entries (26.9%). The GitHub screen alone accepted 8, of which one was a sibling repository for a different paper in the same series. Taken together the two automated signals flag 10 entries, of which 8 are genuine official artifacts and 2 are not — a precision of 80.0% and a recall, against the adjudicated ground truth, of 61.5% (Table 3). After human adjudication, 13 of 26 entries (50.0%) have a public official artifact (Figure 3, Table 4).

Two observations.

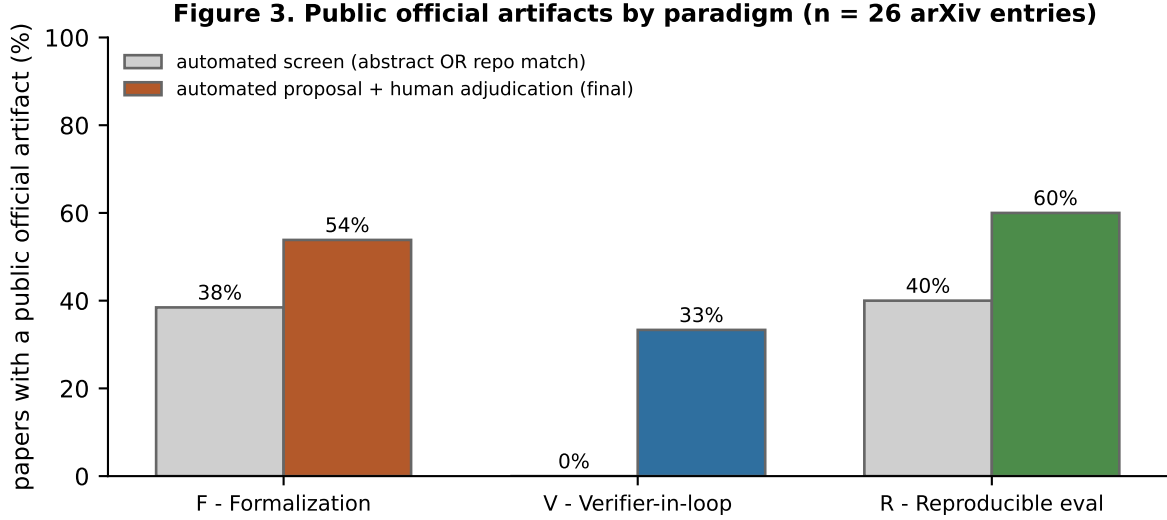


Figure 3: Study B. Share of papers shipping a public official artifact, by paradigm. The automated screen under-detects at every link: five of the thirteen official artifacts were recovered only by human adjudication.

Table 4: Study B by paradigm. “Automated” is the union of the two automated signals from §4; “Adjudicated” is the final figure after all 26 entries were checked by hand. The gap is the method’s false-negative rate.

Paradigm	Measured	Automated	Adjudicated	Missed by automation
F — formalization	13	5 (38%)	7 (54%)	2
V — verifier-in-loop	6	0 (0%)	2 (33%)	2
R — reproducible eval.	5	2 (40%)	3 (60%)	1
Total (paradigm-classified)	24	7	12	5

The missed cases are instructive. The *Program of Thoughts* repository [Chen et al., 2022] states its relationship to the paper in its README rather than its description, and its name differs from the paper’s title by more than the threshold; the GSM8K repository [Cobbe et al., 2021] carries no description at all. Neither is visible to a name-and-description heuristic. This is the framework’s point in miniature: the screen is itself a reduction ($L_1 \rightarrow L_2$, from a repository page to a verdict) and it is precisely that reduction which is unreliable. The repair that actually worked was to add a verification layer — human adjudication against the repository README — not to tune the screen. Improving the screen would have moved the threshold, not removed the residual risk.

Second, on the measured sub-corpus, no paradigm is well served. Even under the most generous reading, half the corpus ships no public official artifact, and the V family — whose entire case rests on empirical comparisons of verifiers — is the least well provided for. We note the irony without over-reading it: the sample is small, and 26 entries selected for a literature review are not a random sample of the field.

6 The reliability checklist

The framework is only useful if it changes what a practitioner does on Monday. Table 5 states it as twelve questions. The first four concern the chain itself and apply to any system; the remainder are paradigm-specific and should be answered for whichever paradigm is in use. The list is intended to be applied *before* a reliability claim is made, not to audit one afterwards.

Table 5: A twelve-item reliability checklist derived from the assurance chain. Each item names the link it interrogates and the failure it guards against.

#	Question	What it guards against	Link
1	What exactly is the claim, written out in one sentence?	An untestable or drifting L_1 .	L_1
2	What is the formal or executable object, concretely?	A reduction that was never performed.	L_2 – L_3
3	Who decided the verdict, and is it independent of the generator?	Self-verification; correlated failure modes.	L_4
4	What is still unverified if this verdict is correct?	Silent assumption that the chain is airtight.	all
5	Is the verifier a model? If so, on what distribution was it trained?	Distributional misalignment of V .	L_4
6	Does the verifier ever disagree with the generator in practice?	A verifier that never disagrees adds nothing.	L_4
7	Was the informal statement checked against the formal one by a human?	Proving the wrong theorem correctly (F).	L_2
8	What fraction of the target mathematics is formalisable at all?	Coverage inflated by choosing easy sub-problems.	L_2
9	Does the benchmark measure the capability, or a proxy for it?	Construct validity (R).	pre- L_1
10	Could the system have seen the test items during training?	Contamination.	pre- L_1
11	Is the benchmark saturated at the current frontier?	A metric that no longer discriminates.	pre- L_1
12	Is the evaluation artefact public, runnable, and pinned?	Claims that cannot be reproduced.	pre- L_1

7 Discussion

7.1 Implications

The practical reading of Table 1 is that reliability work should be organised around residual risk, not around paradigm preference. A deployment that must not assert false theorems will weight F, but only if someone checks the formalisation; a deployment that must not ship a wrong numerical answer among many will weight V, but only if the verifier is external. In both cases the framework’s contribution is negative and specific: it says which risk the chosen paradigm does *not* address, which is the question that is usually left unasked.

The self-referential result in §5.1 generalises beyond bibliographies. Everywhere a language model is used to produce structured artefacts that a downstream process treats as ground truth — citations, schema mappings, test oracles, extracted specifications — the same pattern applies: the artefact is an L_1 object, there is no L_2 reduction, and the only available L_4 is a check by an independent process. The measured rate here, 9.7%, is not a large failure rate, and that is exactly what makes it dangerous: it is low enough to be ignored and high enough to matter.

7.2 Limitations

Study A. The corpus is 31 entries in one domain (AI for mathematics), generated by one model in one session. The 9.7% figure should be read as an existence proof and an order of magnitude, not as a field estimate. The adjudication is subjective in the sense that deciding whether a title difference is an error requires judgement; we mitigate this by recording every

decision with its evidence, so a reader can disagree with a specific call rather than with the aggregate.

Study B. Three limitations are material. First, we measure the existence of a *public official* artifact and say nothing about whether it runs, matches the paper’s claims, or is maintained. Second, four entries were excluded for lack of an arXiv identifier, and these skew towards journal-published work (three *Nature* articles and one LNCS chapter), which plausibly differ in artifact culture. Third, the corpus is a curated literature review, not a random sample, and its composition was chosen to support the framework rather than to test it. The 50% figure should not be quoted as a property of the field.

The framework itself. The four-link decomposition is a modelling choice, and it is not the only reasonable one; one could argue for splitting L_2 into parsing and typing, or for treating resource bounds at L_4 as a separate link. We claim the decomposition is useful, not that it is canonical. The residual-risk matrix likewise asserts that the risks are incommensurable, which is a claim about the absence of a common scale; we would regard a demonstrated common scale as a refutation.

Threats to validity. The single largest threat to this paper is that its own argumentative text is machine-generated. We address it in §7.3 by making the verification chain explicit and by reporting the measured failure rate of the generation step rather than asserting its reliability.

7.3 Provenance and verification statement

This paper was produced in a human-directed workflow in which language models drafted text, proposed the framework, and generated candidate references. In accordance with the framework’s own thesis, the generation steps were paired with independent verification:

1. **Citations.** Every reference in `references.bib` passed the three-source check of §4; the file is generated by script and its SHA-256 is recorded. Three generated references failed and were corrected; the corrections are recorded with their evidence.
2. **Quantitative claims.** Every number in §5 is emitted by `pipeline/03_corpus_audit.py` from archived API responses. No number in this paper was written by hand.
3. **Figure–table consistency.** Figures read their inputs from the same JSON as the tables, so they cannot disagree.
4. **Framework claims.** The attributions in §2 and §3 were checked against the cited abstracts. Where the claim is ours rather than a cited author’s, it is stated as ours (Table 1 is an argument, not a survey result).

What is *not* verified is the argumentative prose. No human has line-by-line audited the reasoning in §3, and readers should treat the residual-risk matrix as a claim to be argued with. We regard this as the honest position: the measurement layer is verified, the argument layer is not, and the paper says so.

7.4 Future work

Three directions follow directly. First, the audit in §5.1 should be repeated across models and domains to turn an existence proof into an estimate. Second, the checklist in Table 5 should be applied prospectively to a system before and after its deployment, which would test whether the residual risks it names are the ones that actually materialise. Third, the framework needs a

formal treatment: the claim that the residual risks are incommensurable is currently an informal one, and a lattice-theoretic ordering of assurance properties would either sharpen it or refute it.

8 Conclusion

We have argued that reliability in AI for mathematics is best understood not as a single property to be maximised but as a chain of four links, and that the three families of mechanisms the field has developed — verifiers, formalization, and reproducible evaluation — each defend a proper subset of that chain while leaving a residual risk of a different kind. The practical consequence is that the useful question is not which paradigm is best but which residual risk a given use can tolerate.

We then applied the framework to our own literature review, and found that its citation-generation step failed at a rate of 9.7%, with the most dangerous failure being a syntactically valid identifier pointing at an unrelated paper. We measured the practice of the evaluation paradigm in the surveyed corpus and found that half of it ships no public official artifact, and that an automated screen recovers only about six in ten of what is there while also flagging things that are not. In both studies the repair was the same: add an independent verification layer, and keep the record of what it caught.

None of this makes language models safe to use for mathematics. It does suggest that the discipline which makes them usable is the same one the field already knows how to practise — reduce, check, and be explicit about what remains unchecked.

References

- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks, 2022. URL <https://arxiv.org/abs/2211.12588>.
- Xiaoyang Chen and Xiang Jiang. Lectures on AI for Mathematics, 2026. URL <https://arxiv.org/abs/2604.11504>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. 2021. doi: 10.1007/978-3-030-79876-5_37. URL https://doi.org/10.1007/978-3-030-79876-5_37.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided Language Models, 2022. URL <https://arxiv.org/abs/2211.10435>.
- Jesse Michael Han, Jason Rute, Yuhuai Wu, Edward W. Ayers, and Stanislas Polu. Proof Artifact Co-training for Theorem Proving with Language Models, 2021. URL <https://arxiv.org/abs/2102.06203>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring Mathematical Problem Solving With the MATH Dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.

- Jie Huang, Xinyun Chen, Swaroop Ranjan Mishra, Huaixiu Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large Language Models Cannot Self-Correct Reasoning Yet, 2023. URL <https://arxiv.org/abs/2310.01798>.
- Albert Qiaochu Jiang, Sean Welleck, Jin Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs, 2022. URL <https://arxiv.org/abs/2210.12283>.
- Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks, 2024. URL <https://arxiv.org/abs/2402.01817>.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Tai Loy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s Verify Step by Step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving, 2025. URL <https://arxiv.org/abs/2502.07640>.
- Chris J. J. Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery, 2024. URL <https://arxiv.org/abs/2408.06292>.
- Stanislas Polu and Ilya Sutskever. Generative Language Modeling for Automated Theorem Proving, 2020. URL <https://arxiv.org/abs/2009.03393>.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A Graduate-Level Google-Proof Q&A Benchmark, 2023. URL <https://arxiv.org/abs/2311.12022>.
- Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Zhenhua Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition, 2025. URL <https://arxiv.org/abs/2504.21801>.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, Manish Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 2023. doi: 10.1038/s41586-023-06924-6. URL <https://doi.org/10.1038/s41586-023-06924-6>.
- Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier López de Lacalle, and Eneko Agirre. NLP Evaluation in trouble: On the Need to Measure LLM Data Contamination for each Benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. doi: 10.18653/v1/2023.findings-emnlp.722. URL <https://doi.org/10.18653/v1/2023.findings-emnlp.722>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Răileanu, María Lomelí, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools, 2023. URL <https://arxiv.org/abs/2302.04761>.

- Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the Self-Verification Limitations of Large Language Models on Reasoning and Planning Tasks, 2024. URL <https://arxiv.org/abs/2402.08115>.
- Trieu H. Trinh, Yuhuai Wu, Quoc Viet Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 2024. doi: 10.1038/s41586-023-06747-5. URL <https://doi.org/10.1038/s41586-023-06747-5>.
- Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the Planning Abilities of Large Language Models : A Critical Investigation, 2023. URL <https://arxiv.org/abs/2206.10498>.
- Hanchen Wang, Tianfan Fu, Yuanqi Du, Wenhao Gao, Kexin Huang, Ziming Liu, Payal Chandak, Shengchao Liu, Peter Van Katwyk, Andreea Deac, Anima Anandkumar, Karianne J. Bergen, Carla Pedro Gomes, Shirley Ho, Pushmeet Kohli, Joan Lasenby, Jure Leskovec, Tie-Yan Liu, Arjun Kumar Manrai, Debora S. Marks, Bharath Ramsundar, Le Song, Jimeng Sun, Jian Guo Tang, Petar Veličković, Max Welling, Linfeng Zhang, Connor W. Coley, Yoshua Bengio, and Marinka Žitnik. Scientific discovery in the age of artificial intelligence. *Nature*, 2023. doi: 10.1038/s41586-023-06221-2. URL <https://doi.org/10.1038/s41586-023-06221-2>.
- Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddhartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. LiveBench: A Challenging, Contamination-Limited LLM Benchmark, 2024. URL <https://arxiv.org/abs/2406.19314>.
- Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with Large Language Models, 2022. URL <https://arxiv.org/abs/2205.12615>.
- Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data, 2024. URL <https://arxiv.org/abs/2405.14333>.
- Kaiyu Yang, Aidan Swope, Alex Gu, R Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models, 2023. URL <https://arxiv.org/abs/2306.15626>.
- Kun Zhou, Yutao Zhu, Zhipeng Chen, Wen-Tong Chen, Wayne Xin Zhao, Chen Xu, Yankai Lin, Ji-Rong Wen, and Jiawei Han. Don’t Make Your LLM an Evaluation Benchmark Cheater, 2023. URL <https://arxiv.org/abs/2311.01964>.

A Audit trail

The complete audit trail is distributed with the paper rather than reproduced here. The following files are the primary evidence for every empirical claim above.

File	Contents
pipeline/refs_candidates.json	The v2 (corrected) candidate list. v1, the raw memory-written list, is preserved in the verification archive.
pipeline/reports/citation_verification_v1_from_memory.json	The complete v1 verification run: for each candidate, the top five hits from each of the three indexes, with similarity scores. Unmodified.
pipeline/refs_corrections.json	Every v1→v2 correction with its severity, the wrong value, the corrected value, and the evidence that decided it.
pipeline/reports/citation_verification.md	Per-entry verification report: title similarity, author and year agreement, matched title, venue, DOI, and which index confirmed it.
pipeline/artifact_adjudication.json	Study B's manual adjudication: tier, repository, and the evidence for all 26 entries.
pipeline/reports/corpus_audit.json	Both studies' machine-readable output, including the raw artefact-availability rows and the GitHub query results.
pipeline/reports/references_bib_sha256.txt	SHA-256 of the generated bibliography, allowing byte-exact reproduction.